Thomas Parisi
ITS - Commit Discipline & Conventions

Domain (Expert) Model

Efficiently utilize the Git CLI to stage, commit, and describe changes using appropriate commit messages

- What “staging” means in git (staging area)
- What a commit is and what it represents (snapshot of history)
- What is the working directory, staging area, and remote repository are and how they differ
- The difference between staged and unstaged files
- Stage specific files with ‘git add <file>’
- Stage all unstaged files with ‘git add .’ (and when not to) – maybe don't teach this
- Create a commit with ‘git commit -m <message>’
- Distinguish between ‘modified but unstaged’ and ‘staged but un-committed’
- Decide which files should NOT be staged (.env, node_modules), which files belong in .gitignore
- What is a .gitignore file and how does it affect the staging area
- Choose between ‘git add <file>’ and ‘git add .’ –maybe dont teach

Recognize which file(s) need to be committed

- The difference between tracked and untracked files
- Run ‘git status’ to check current repository state
- Interpret ‘git status’ output to identify which files to stage
- Decide which files should be staged vs ignored
- (misconception) Belief that git status shows what is committed, not what is unstaged/staged

Unstage a file using git restore --unstage <file>

- Unstage a file using ‘git restore --unstage <file>’
- (misconception) Belief that ‘git restore <file>’ does the same thing as ‘git restore --unstage <file>

Identify what a good commit message looks like

- What a good commit message is and its purpose for future readers
- Why commit messages must be human-readable and searchable
- Craft a subject line commit message under 50 characters
- Use imperative mood in commit messages (“Fix” not “Fixed”)
- Write a multi-line commit message with subject + body when appropriate
- Identify when a commit message is too vague vs appropriately descriptive
- (misconception) belief that commit messages are only for yourself
- (misconception) belief that a good commit message just says what you did, not why

Summarize changes/additions in a concise commit message

- What a commit message is and its purpose for future readers
- Craft a subject line commit message under 50 characters
- Use imperative mood in commit messages (“Fix” not “Fixed”)
- Write a multi-line commit message with subject + body when appropriate
- Identify when a commit message is too vague vs appropriately descriptive

Identify when to commit / demonstrate regular focused commits

- The concept of ‘atomic commits’ – one logical change per commit
- Why committing too infrequently is risky (large, hard to revert changes)
- Why committing too frequently with trivial messages pollutes history
- Decide when a set of changes is complete enough to commit
- Recognize that a commit should represent a single logical unit of work
- Recognize when to break a large change into multiple smaller commits
- (misconception) belief that ‘git add .’ is always safe to use before committing
- (misconception) belief that committing often means committing every single file save

Thomas Parisi
ITS - Commit Discipline & Conventions

Student Model

Conceptual knowledge
represented by: state level (novice, partial, mastery)
assessed with: multiple choice or true or false questions:

- What is the staging area
- What a commit represents (snapshot)
- difference between working dr, staging area, remote
- What a .gitignore file is and how it works
- What a good commit message is and its purpose
- What atomic commits are
- Why commit frequency matters

Procedural Knowledge
Represented by: decimal
How assessed: open ended / given scenario, how would you react

- Run git status and interpret its output
- Use git add <file> to stage a specific file
- Use git commit -m <message> to commit
- Unstage a file using git restore –staged <file>
- Craft a subject line commit message under 50 char
- Use imperative mood in commit messages
- Write a multi-line commit message (subject + body)

Conditional Knowledge
Represented by: state level (novice, partial, mastery)
How assessed: given scenario, how would you react

- Decide which files should NOT be staged
- Divide which files belong in .gitignore
- distinguish between too-vague vs appropriately descriptive commit messages
- Recognize when a change is complete enough to commit
- Recognize when to break a large change into multiple commits

Misconceptions
Represented by: boolean

- Believes git status shows what is committed, not what is staged/unstaged
- Believes ‘git restore <file>’ is the same as ‘git restore –staged <file>’
- Believes commit messages are only for yourself
- Believes a good commit message only describes *what,* not *why*
- Believes ‘git add .’ is always safe to use before committing
- Believes committing often means committing after every file save

Thomas Parisi
ITS - Commit Discipline & Conventions

Tutoring Model

The system will use a hint ladder for feedback. When a student answers incorrectly, they receive progressively more specific hints before the answer is revealed as a last resort. When a misconception flag is triggered, the system finishes the current question, then inserts a targeted intervention before presenting the next question.
Progression can adapt based on the students state level per question (novice, partial, mastery)
Hint Ladder:

Hint 1: General nudge/problem reframe
Hint 2: specific clue pointing to the answer
Hint 3: near-answer, almost giving it away
Revealed: full answer with explanation

The reveal will be gated by multiple incorrect answers after hint 3 so the student can not easily bypass to the full answer. Also, if hints were required, the state level change for that given KC will increase more slowly
Misconception Interventions:

When a misconception flag is triggered, the system will output a pre-written explanation for that given misconception after the question is completed. This way, the next time a similar question comes along the user will have seen that misconception feedback.
Ex:
Misconception: status shows committed files
If the student selects an option that implies git status reflects commit history they receive:
“ Quick note: git status only shows the current state of your working directory and staging area, it does not say anything about what has already been committed. To see commit history, you’d use ‘git log’ “

Adaptive Progression

Novice
- Questions focused on conceptual knowledge first
- No fill in the blank until some conceptual components reach ‘partial’ status
- Full 3 hint ladder always available

Partial
- Conceptual questions less frequent, procedural and conditional questions more prevalent
- Git status interpretation scenarios become more complex
- Hint ladder still available with tracking on how often hints are requested, slowing progress
- Commit message fill-in-the-blank questions introduced with code change scenarios

Masterey
- Questions focus on "given this scenario what would you do”

Thomas Parisi
ITS - Commit Discipline & Conventions

- Multi step scenarios, “given git status output, what do you stage and what is the commit message”
- Hint ladder available with tracking
- Critical thinking edge case presented

Thomas Parisi
ITS - Commit Discipline & Conventions

User Interface (wireframe)

Interpret git status | Procedural

KC: run_and_interpret_status · score 0.61

Read the `git status` output below. Which answer correctly describes the state of each file?

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified: index.js
Changes not staged for commit:
  (use "git add <file>..." to update)
    modified: App.css
Untracked files:
  (use "git add <file>..." to include)
    .env
```

a) index.js is staged; App.css is unstaged; .env is untracked

b) All three files are staged and ready to commit

c) index.js is committed; App.css and .env need to be staged

d) index.js is unstaged; App.css is staged; .env is untracked

Hint ladder (0/3 used) | Request hint →

Submit

Fill in the blank | Procedural

KC: unstage_file · score 0.38

You accidentally staged `App.css` and want to remove it from the staging area *without* discarding your edits. Write the exact command:

```
$ type your command here...
```

Submit

Thomas Parisi
ITS - Commit Discipline & Conventions

CONCEPTUAL KNOWLEDGE

What the staging area is	Mastery
What a commit represents	Mastery
Working dir / staging / remote differences	Partial
What .gitignore does	Partial
What a good commit message is	Novice
Atomic commits concept	Novice
Commit frequency tradeoffs	Novice

PROCEDURAL KNOWLEDGE

Run & interpret git status	61%
git add <file>	78%
git commit -m <message>	55%
git restore --staged <file>	38%
Craft subject-line commit message	42%
Imperative mood in messages	30%
Multi-line commit message	10%

CONDITIONAL KNOWLEDGE

Decide what NOT to stage	Partial
Identify .gitignore candidates	Novice
Vague vs. descriptive messages	Partial
Judge when to commit	Novice
Recognize when to split commits	Novice

MISCONCEPTION FLAGS

- ☐ git status shows committed files
- ☒ **git restore = git restore --staged** triggered
- ☐ Commit messages are only for yourself
- ☐ Good message only says what, not why
- ☐ git add . is always safe
- ☐ Committing often = committing every save